

Nature's Notes

Team 5

Will Hedges

Ruth Higgason

Benjamin Kozlowski

Eric Loseke

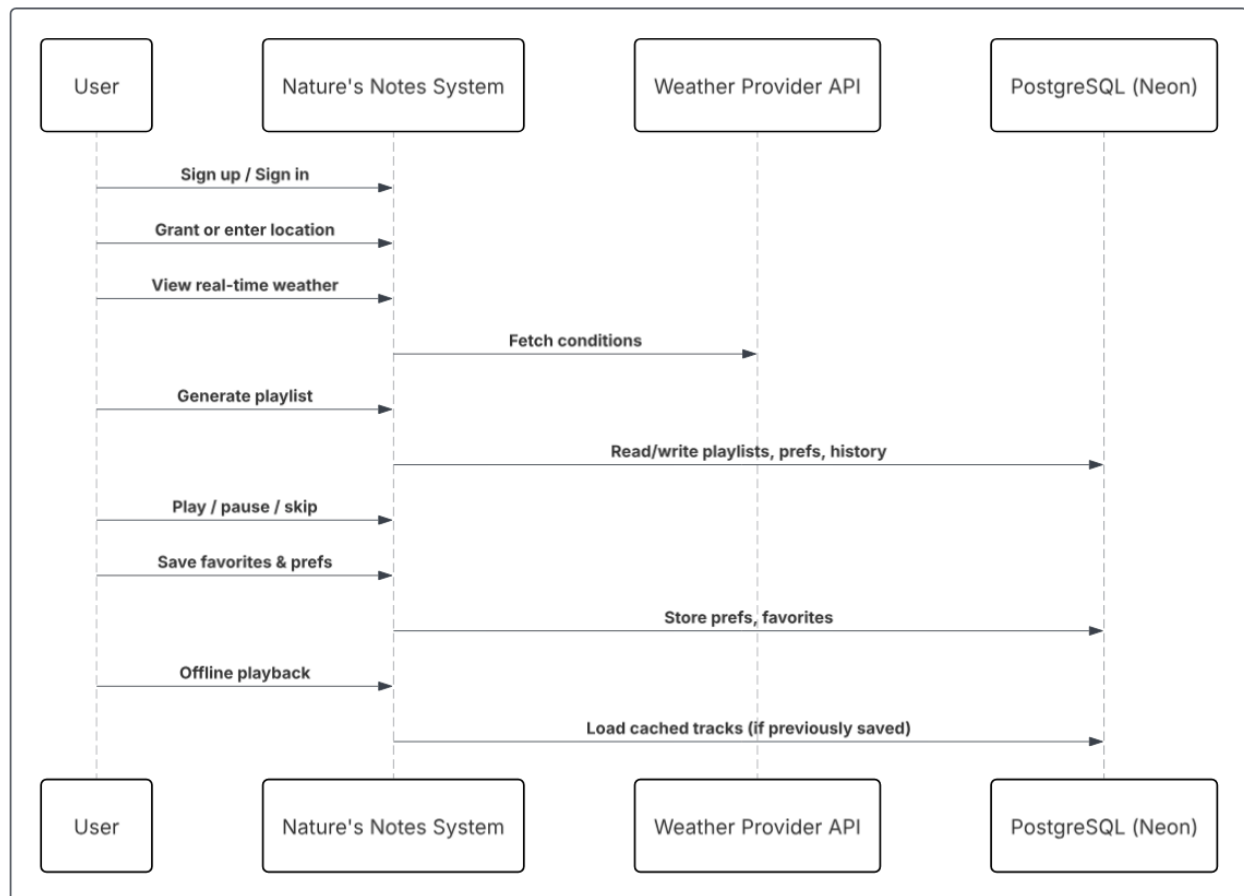
MJ McGee

Synopsis

A weather-music web app built with React, Node.js, and Postgres, that captures user-location, fetches live weather data, and generates playlists based on the conditions.

Architecture Description

This weather-music web application is built on top of the React framework. It will follow a client-server model. The application will get the user's location from entering it manually in the UI entry field, or also by accepting web location permissions. This location will be passed to the API, theweatherapi.com or another similar API, through an HTTPS query. Then it will reply and give weather data based on the user's location. For storing user data and customization, we will use a node.js backend to communicate with a Postgres database through the cloud platform Neon.



UML#1: Use Case Diagram

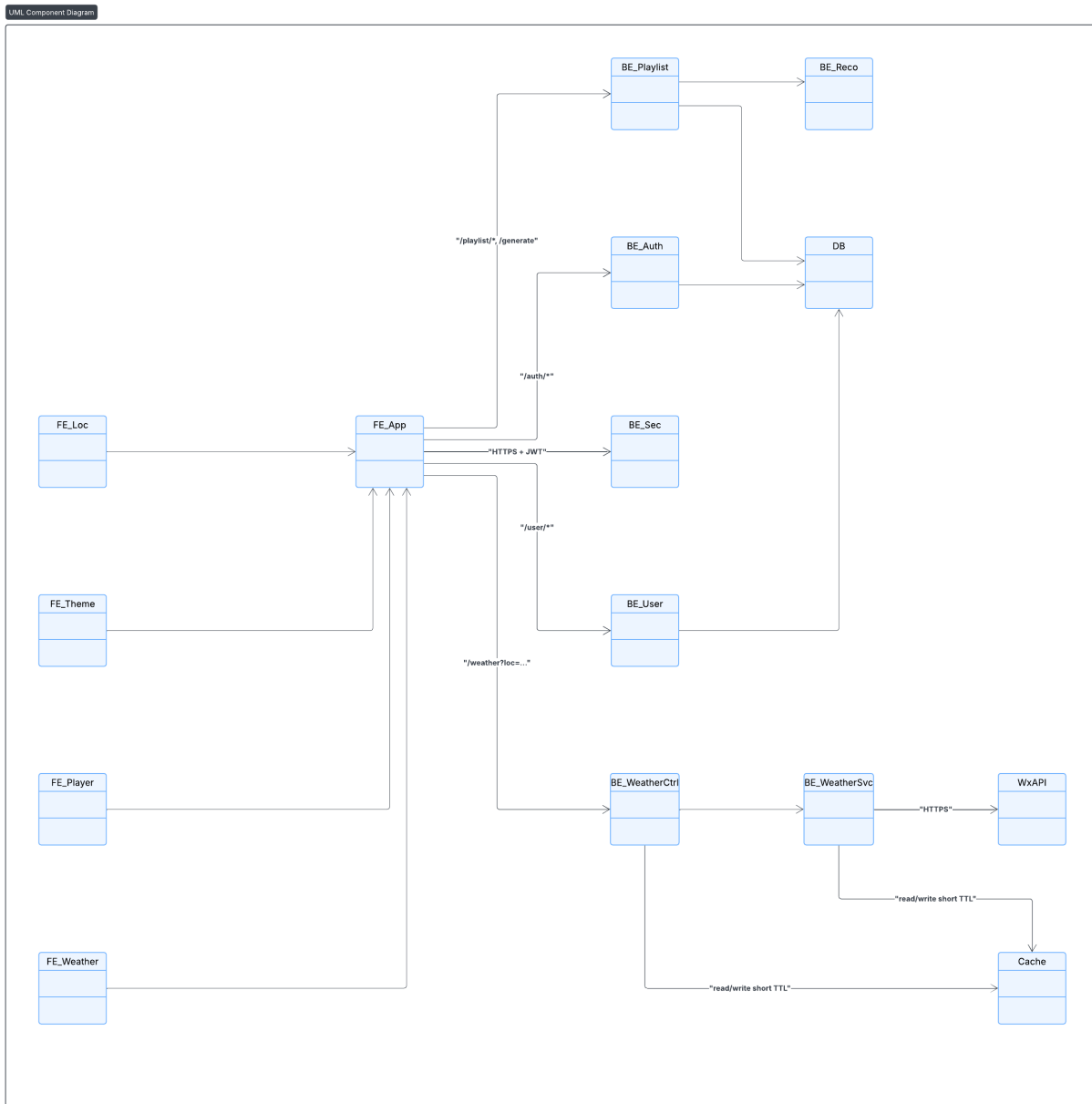
-This shows what the system does from the user's perspective.

-Actors: user, weather provider API, database.

-Core interactions: sign in, enter location, fetch weather, generate playlist, play music, store preferences.

-External integrations shown as separate systems

1. Front-end Architecture



UML #2: Component Diagram

-This shows the main building blocks and how they connect.

-Frontend: React components for location entry, weather display, player, and theme.

-Backend: Node controllers and services.

- Database: PostgreSQL (Neon).
- External API: Weather provider.
- Emphasizes data flow through HTTPS and internal service calls.

The front end is implemented with React, which makes it component-based in structure. The application allows starting with a basic page setup design that goes with a minimalistic UI to keep the focus on the content and not on the interface. Key UI components include:

- **Location Entry & Selector:** Allows users to either grant location or manually input the city.
- **Weather Display Widget:** It shows real-time weather conditions fetched from third-party weather API.
- **Audio Player Component:** Provides playback controls for start, pause, and skip.
- **Visual theme layer:** Runtime changes of visuals, animations, and themes depending on weather conditions and time of day.

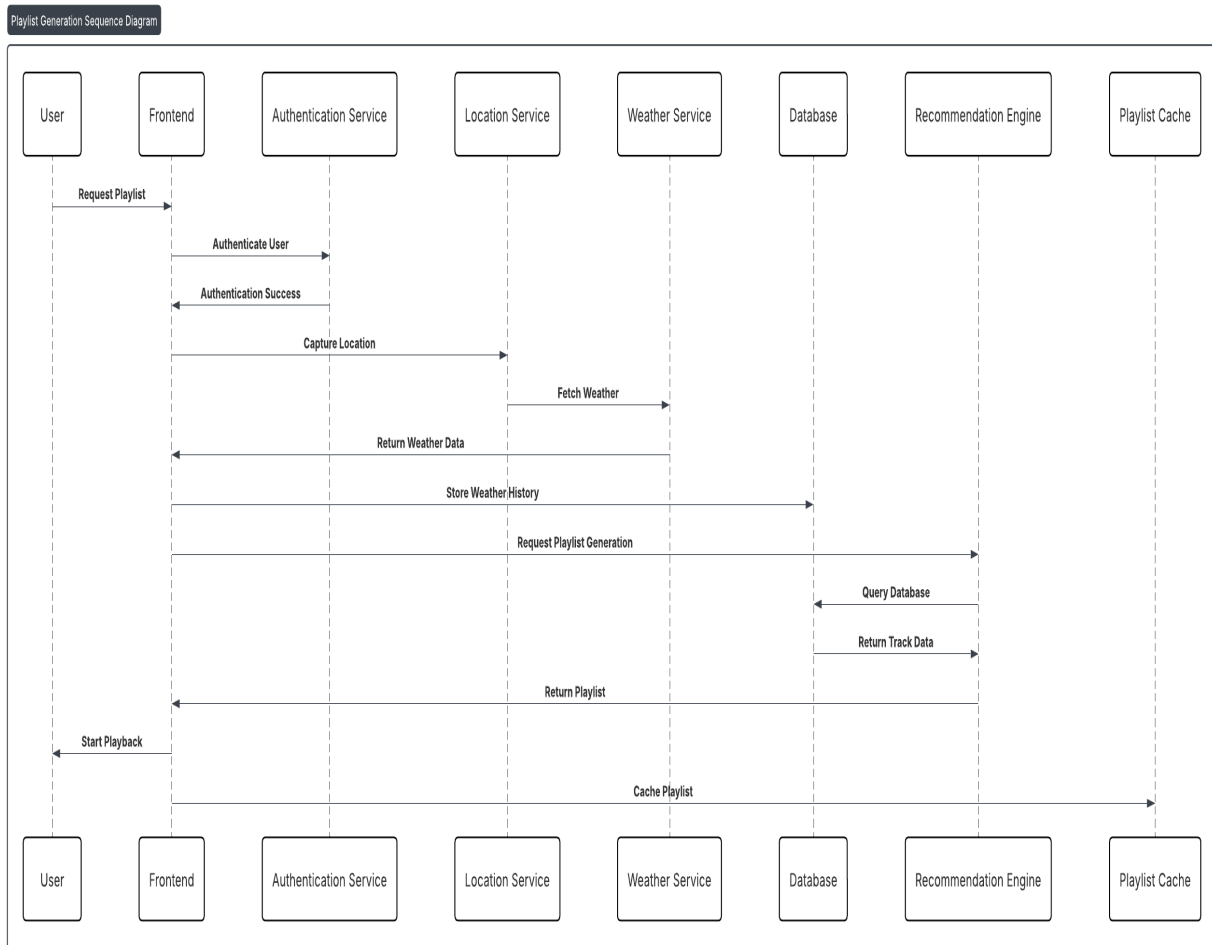
Everything communicates through React state management and context to make sure that if one thing changes, location, weather, then the UI will also change.

2. Backend Architecture

Business logic, user account management, playlist generation, and data security are handled in the backend layer. The idea is to have a RESTful service, using Node.js.

Key backend roles:

- **User Management:** Handles authentication, recent locations, preferred moods, and favorite music types.
- **Weather Synchronization:** It fetches and updates automatically on the user-selected locations.
- **Music Recommendation Engine:** This matches weather data with pre-made playlists or automatically generated tracks.
- **Offline Mode Handling:** The ability to play saved playlists without active weather data.



UML #3: Sequence Diagram This shows runtime flow of a playlist generation scenario.

- Starts with authentication.
- Captures location and fetches weather.
- Stores weather history.
- Generates playlist using recommendation engine and database.
- Returns tracks to frontend and starts playback.
- Caches playlist for offline use.

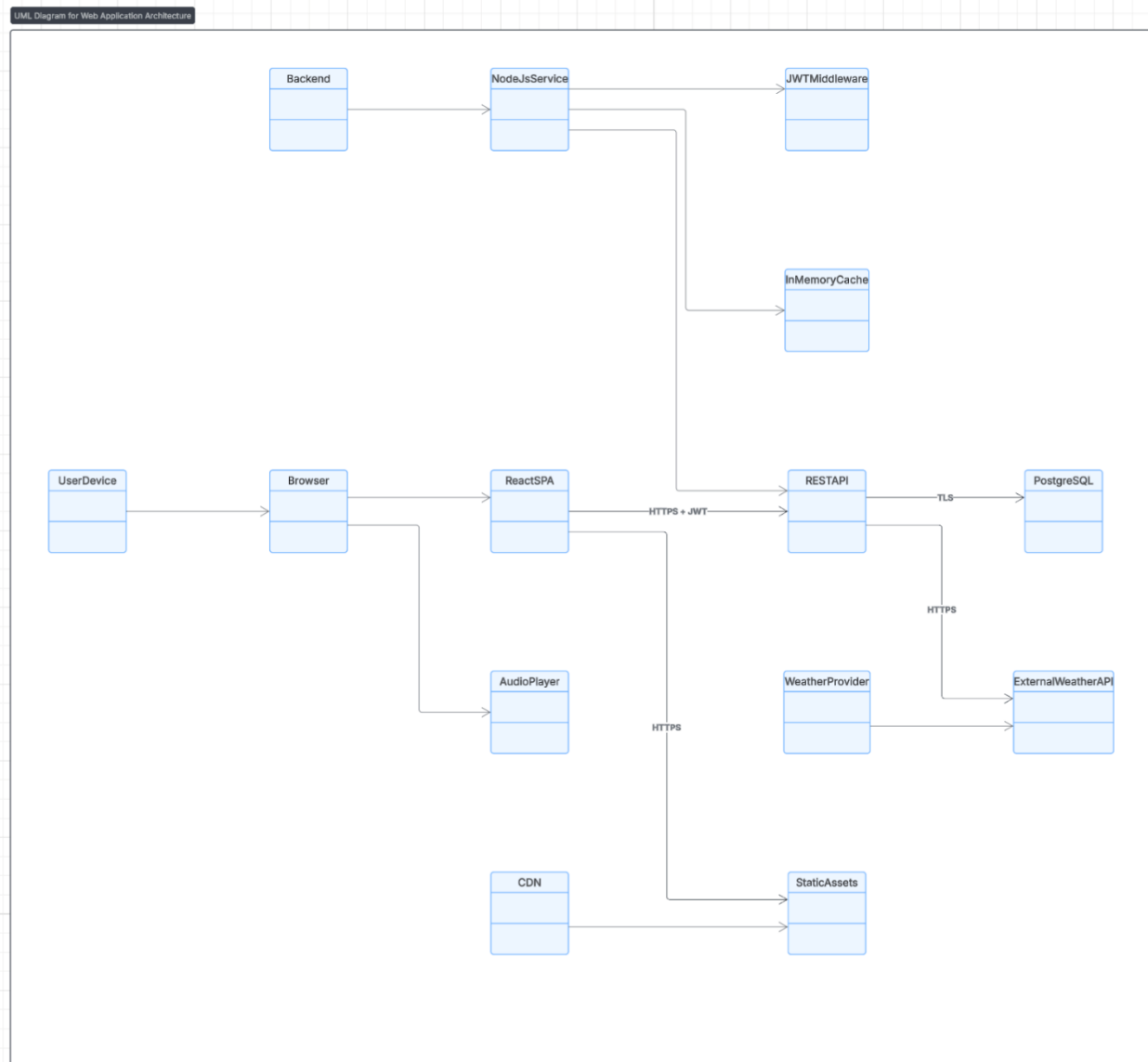
3. Database Layer

Database PostgreSQL hosted via Neon, providing a cloud relational database. The schema includes tables for:

- Users and authentication information
- Saved places and weather history
- Playlist metadata and user preferences
- Preferences such as setting mood and themes

4. External Integrations

The system integrates the Weather API, OpenWeather or other, to fetch real-time data depending on user location permissions or manual input. The music can be provided through premade playlists that correspond to different weather conditions such as sunny, windy, rainy, and cloudy.



UML #4

Deployment Diagram: This shows where the parts run.

- User device runs React and audio player.
- CDN serves static assets. In this case, our project will use Vercel as our CDN.
- Node backend hosts API, business logic, and cache.
- PostgreSQL Neon is the cloud database.
- Weather provider is an external service through OpenWeatherMap

5. Safety and Privacy

Security of data is considered all the way up the stack. All communication between the frontend and backend will be with HTTPS. And for user authentication, we will use token-based authentication. Sensitive data includes storing user locations and preferences in the database. No personal data will be shown publicly.

Conclusion

This architecture uses React for UI, a Node.js backend for business logic, a Postgres database for user data storage in the cloud platform Neon, and external APIs for real-time weather and integration of music.